SecureMailAnalyzer

Secure Email Attachment Analysis and Threat Investigation Platform

---

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

TEAM NAME:SPARKFEST

Date of preparation: 10 October 2026

CONTENTS

1.How to Read the Verification Labels
2. Abstract
3. Introduction
4. Problem Statement
5. Project Objectives
6. Existing System and Its Limitations
7. Proposed System
8. Project Scope
9. Users and Roles
10. Functional Requirements
11. Non-Functional Requirements
12. Technology Stack
13. System Architecture
14. System Modules
15. System Workflow
16. Use Cases
17. Input and Output Requirements
18. Risk Scoring and Classification
19. Attachment Threat Story and Evidence Sources
20. Security and Privacy
21. Database and Data Management
22. User Interface Requirements
23. Error Handling and Validation
24. Testing and Test Cases
25. Software and Hardware Requirements
26. Limitations
27. Future Enhancements
28. Expected Outcomes
29. Conclusion
30. Glossary

1. How to Read the Verification Labels

This SRS was prepared from the project description supplied by the team and from earlier SRS drafts. The source code, the database file and real test results were not available. For this reason, the document does not claim that a feature works unless it can be confirmed. These labels are used:

Label	Meaning
To verify	The feature or detail is described in the project description, but it has not yet been confirmed in the code or by running a test.
Required rule	A safety or quality rule that the application must follow. It is a requirement, not a claim that it is already met.
Future work	A feature that is planned or possible, but is not part of the current application.

2. ABSTRACT

Email attachments can hide harmful code, suspicious scripts or other signs of a cyberattack. Most users cannot tell whether an attachment is safe just by looking at it.

SecureMailAnalyzer is a web-based cybersecurity application that helps users inspect email attachments without running them. It checks file details, looks for suspicious signs, assigns a risk level, and explains the reasons behind the result. It can also read an existing sandbox report in JSON format, show an investigation timeline called the Attachment Threat Story, simulate a quarantine decision, and produce a security report.

The main benefit is clear understanding. The application does not only show a risk score. It shows why a file was marked suspicious and where each piece of evidence came from.

3. INTRODUCTION

3.1 Purpose of the SRS

This SRS explains the requirements of SecureMailAnalyzer. It helps the project team, reviewers and judges understand what the application should do and how it should be tested.

3.2 Project Background

Attackers often send harmful files by email, disguised as invoices, reports or letters. Security teams inspect such files safely, but students and small teams often lack advanced tools. SecureMailAnalyzer is a hackathon project that shows how safe attachment inspection and clear explanations can work together.

3.3 Scope, Users and Goals

The application lets a user upload a supported attachment or import a sandbox JSON report, then shows findings, a risk level, a timeline, a quarantine simulation and a report (full scope in Section 8). Intended users are students and learners, security analysts (for demonstration) and hackathon reviewers.

Main goals: inspect attachments safely, explain results clearly, show where each piece of evidence comes from, keep a record of investigations, and never claim more than the application can do.

4. PROBLEM STATEMENT

Email is one of the most common ways to deliver malware, which is software made to cause harm. Harmful attachments can contain scripts, hidden content, disguised file types or other warning signs.

Users face three main problems:

- They cannot safely tell if an attachment is harmful, because opening it may cause damage.
- Simple tools often give a verdict without a reason, so users do not learn what is wrong.
- Evidence from different sources is often mixed together, so it is hard to know what was really found in the file.

Users therefore need a tool that inspects attachments without running them, shows suspicious signs in plain language and clearly separates real findings from imported or demonstration data.

5. PROJECT OBJECTIVES

1. Inspect supported email attachments safely, without executing them.
2. Calculate a risk score and risk level based on the implemented rules.
3. Explain which indicators caused the result.
4. Import and read supported sandbox JSON reports.
5. Show an investigation timeline and make the source of every event clear.
6. Simulate a quarantine decision without changing any real file.
7. Keep an investigation history, where implemented.
8. Generate a security report, where supported.

6. EXISTING SYSTEM AND ITS LIMITATIONS

Basic attachment checking is often done by hand. Users may judge a file by its name or icon, trust the file extension without checking the content, or use tools that give only a label such as "safe" or "unsafe" with no reason. Real and simulated information can also be mixed, which can mislead the reader.

A tool that combines file inspection, clear findings and a visible evidence source can help users understand a result and decide what to do. This SRS does not compare SecureMailAnalyzer with any specific commercial product.

7. PROPOSED SYSTEM

SecureMailAnalyzer addresses these problems in the following ways. All features below are described in the project description and are marked To verify until confirmed in the code.

Feature	How it helps
Safe file inspection	Checks a file without running it (static analysis).
File details	Calculates a SHA-256 hash and compares the file signature with the extension where supported.
Indicators	Looks for suspicious signs in supported file formats.
Risk assessment	Gives a risk score and a risk level with reasons and recommendations.
Sandbox JSON import	Reads an existing sandbox report and extracts supported indicators.
Attachment Threat Story	Shows events in time order, each with its source.
Quarantine simulation	Shows a simulated decision and reason. No real file is isolated.
Investigation history	Stores and shows earlier investigations.
Security reports	Creates a report of the findings.
Dashboard	Shows statistics and recent activity.

8. PROJECT SCOPE

8.1 What the Application Does

- Accepts supported attachments and sandbox JSON reports
- Inspects files without executing them
- Shows findings, risk levels, timeline events and quarantine simulation results
- Stores investigation records and creates reports, where implemented

8.2 What the Application Does Not Do

- It does not run, open or detonate uploaded files.
- It does not perform a real sandbox analysis. Importing a sandbox report only reads a report that already exists.
- It does not isolate files from the operating system or the network. Quarantine is a simulation.
- It does not guarantee that every harmful file will be found.
- It does not replace professional antivirus, email security or malware analysis products.

9. USERS AND ROLES

The project description does not mention a login system or separate user roles. This SRS therefore describes one type of user and does not add an administrator role.

User	Description	Main actions	Status
User (analyst or student)	A person who inspects attachments and reviews results.	Upload files, import reports, review findings, timeline, quarantine simulation, history and reports.	To verify

If login or roles exist in the application, they should be added here after checking the code.

10. FUNCTIONAL REQUIREMENTS

Priority: Must Have = needed for the core purpose. Should Have = important but not essential. Could Have = useful if time allows. **Verification:** see Section 1.1.

10.1 Dashboard

ID	Requirement	Priority	Verification
FR-01	The system shall show investigation statistics.	Should Have	To verify
FR-02	The system shall show recent activity and risk summaries.	Should Have	To verify
FR-03	The system shall let the user move easily to the other modules.	Must Have	To verify

10.2 File Upload

ID	Requirement	Priority	Verification
FR-04	The system shall allow supported attachments to be uploaded.	Must Have	To verify
FR-05	The system shall check the file type and file size. The supported types and the size limit must be confirmed from the code.	Must Have	To verify
FR-06	The system shall reject empty, unsupported and oversized files and show a clear message.	Must Have	To verify
FR-07	The system shall use safe filenames and prevent unsafe file paths.	Must Have	To verify

10.3 Static File Analysis

Static analysis means checking a file without running it.

ID	Requirement	Priority	Verification
FR-08	The system shall calculate the SHA-256 hash of an uploaded file.	Must Have	To verify
FR-09	The system shall compare the file signature with the file extension where supported.	Must Have	To verify
FR-10	The system shall check supported file types (such as PDF, Office or archive files) for suspicious signs. The exact indicators must be confirmed from the code.	Must Have	To verify
FR-11	The system shall show the findings in a clear and simple way.	Must Have	To verify

10.4 Risk Assessment

ID	Requirement	Priority	Verification
FR-12	The system shall calculate a risk score using the implemented rules.	Must Have	To verify

ID	Requirement	Priority	Verification
FR-13	The system shall assign a risk level (Low, Medium, High or Critical) if supported.	Must Have	To verify
FR-14	The system shall explain which indicators affected the result.	Must Have	To verify
FR-15	The system shall show suitable recommendations based on the findings.	Should Have	To verify

10.5 Attachment Threat Story

ID	Requirement	Priority	Verification
FR-16	The system shall show investigation events in time order.	Should Have	To verify
FR-17	The system shall explain what was found and when it was recorded.	Should Have	To verify
FR-18	The system shall label each event as coming from static analysis, an imported sandbox report or synthetic demonstration data.	Must Have	To verify
FR-19	The system shall never present simulated events as real events seen in the uploaded file.	Must Have	Required rule

10.6 Sandbox JSON Analysis

ID	Requirement	Priority	Verification
FR-20	The system shall allow supported sandbox JSON reports to be imported.	Must Have	To verify
FR-21	The system shall check the report structure and handle invalid input.	Must Have	To verify
FR-22	The system shall extract supported process, network and file activity indicators.	Must Have	To verify
FR-23	The system shall explain what the report shows and state that importing it does not mean the attachment was run by SecureMailAnalyzer.	Should Have	To verify

10.7 Quarantine Simulation

ID	Requirement	Priority	Verification
FR-24	The system shall allow the user to review or simulate a quarantine decision.	Should Have	To verify
FR-25	The system shall show the quarantine status and the reason.	Should Have	To verify
FR-26	The system shall state clearly that simulated quarantine does not isolate a file from the operating system or network.	Must Have	Required rule

10.8 Investigation History

ID	Requirement	Priority	Verification
FR-27	The system shall store investigation records in the database.	Should Have	To verify
FR-28	The system shall show previous investigations to the user.	Should Have	To verify

10.9 Security Reports

ID	Requirement	Priority	Verification
FR-29	The system shall generate a security report for an investigation.	Should Have	To verify
FR-30	The system shall offer only the export formats that are confirmed to work. The supported formats must be confirmed from the application.	Should Have	To verify
FR-31	The report should contain the file details, hash, risk level, findings, evidence sources, quarantine status and a notice about the limits of the analysis.	Should Have	To verify

11. NON-FUNCTIONAL REQUIREMENTS

These statements are written so that they can be tested. No performance numbers are given because none have been measured.

Category	Requirement
Security	The application shall treat every uploaded file and report as untrusted and shall never run uploaded files (see Section 20).
Ease of use	A user should be able to upload a file and read the result without special training. Messages should use plain language.
Reliability	If an analysis fails, the application should show an error message and continue to work for other requests.
Performance	The application should show results without unreasonable delay for the supported file sizes. Time targets have not been defined and should be measured and added later.
Maintainability	Code should be organised into separate modules (for example analysis, risk scoring and reports) so rules and checks can be updated.
Data integrity	Stored investigation records should match the results shown to the user. The SHA-256 hash shown must match the file that was analysed.
Input validation	File type, file size, filename and JSON structure shall be checked before processing.
Error handling	Errors shall be shown in clear messages and must not reveal internal details or crash the application (see Section 23).
Privacy	Uploaded files and results should stay on the local system unless the user chooses to export them. Retention details must be confirmed.

Category	Requirement
Portability	The application should run on a standard computer with the required Python and Node.js versions. Windows is the confirmed development environment.

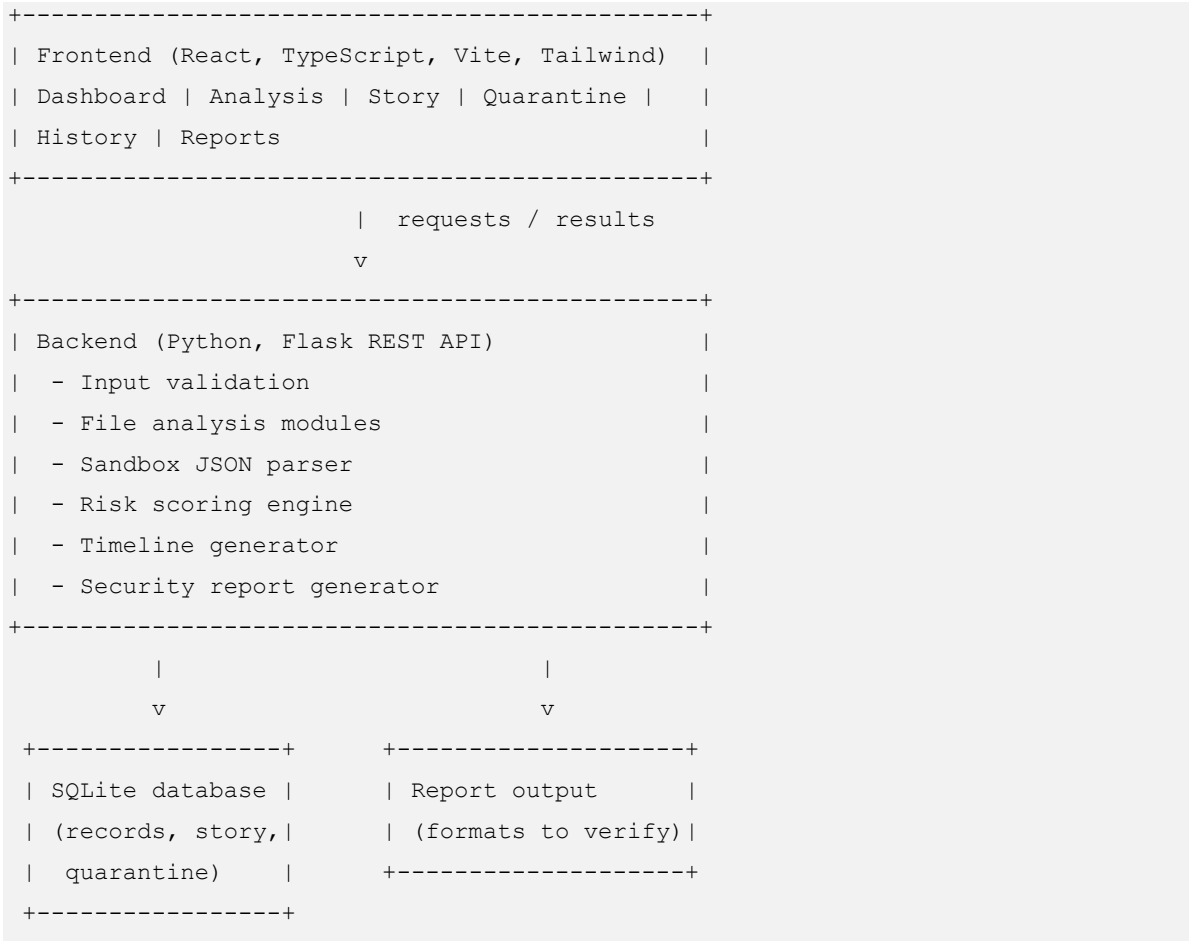
12. TECHNOLOGY STACK

The technologies below come from the project description. The exact packages and versions should be confirmed from the project files. Technologies that are not listed here, such as YARA, cloud services, machine learning or real sandbox software, are not claimed as part of the current application.

Technology	Role in the project	Status
React	Builds the user interface.	To verify
TypeScript	Adds type checking to the frontend code to reduce mistakes.	To verify
Vite	Builds and runs the frontend during development.	To verify
Tailwind CSS	Styles the interface.	To verify
Recharts, Lucide icons	Draw charts and icons, where used.	To verify
Python	Runs the backend and the analysis code.	To verify
Flask REST API	Receives requests from the frontend and sends back results.	To verify
Flask-CORS	Allows the frontend and backend to talk to each other, if used.	To verify
File analysis modules	Inspect files and calculate hashes and indicators.	To verify
Risk scoring engine	Calculates the risk score and level.	To verify
Sandbox JSON parser	Reads imported sandbox reports.	To verify
Timeline generator	Builds the Attachment Threat Story.	To verify
Report generator	Creates security reports. Formats must be confirmed.	To verify
SQLite	Stores investigation data where implemented.	To verify
VS Code, Windows PowerShell	Tools used for development.	To verify
Automated tests, safe sample data	Used to test the application safely.	To verify

13. SYSTEM ARCHITECTURE

The diagram below shows how the main parts work together. It uses only the components named in the project description. Component names and connections should be confirmed from the code.



Step-by-step explanation

1. The user works in the web interface built with React.
2. The interface sends the uploaded file or JSON report to the Flask backend.
3. The backend checks the input. Invalid input is rejected with a message.
4. The file analysis modules inspect the file as data. They never run it. The JSON parser reads imported reports.
5. The risk scoring engine evaluates the indicators and sets the risk level.
6. The timeline generator builds the Attachment Threat Story and keeps the source of each event.
7. The results are saved in the SQLite database, where implemented.
8. The backend sends the results to the interface, and the report generator creates a report when requested.

14. SYSTEM MODULES

Each module below is described in the project description. Inputs, processing and outputs are general and should be confirmed against the code.

Module	Purpose	Input	Processing	Output
Dashboard	Give an overview	Stored investigation data	Counts and summarises records	Statistics, recent activity, risk summary
File upload	Receive attachments	Attachment file	Checks type, size and filename	Accepted file or error message
Static file analysis	Inspect a file safely	Accepted file	Calculates SHA-256, checks signature and indicators	Findings
Risk assessment	Judge the risk	Findings	Applies the implemented rules	Risk score, level, reasons, recommendations
Sandbox JSON analysis	Read existing reports	JSON report	Checks structure, extracts indicators	Indicators and explanation
Attachment Threat Story	Show events in order	Findings and report events	Orders events and keeps the source label	Timeline
Quarantine simulation	Show a simulated decision	Risk result	Selects a status and reason	Quarantine status
Investigation history	Keep past results	Completed investigations	Saves and reads records	List of investigations
Security reports	Create reports	Investigation data	Builds the report	Report file

15. SYSTEM WORKFLOW

The main workflow is shown in Figure 1. It should be adjusted if the real application works differently.

1. The user opens SecureMailAnalyzer.
2. The user uploads a supported attachment or imports a sandbox JSON report.
3. The system checks the input.
4. The system inspects the file without running it.
5. The system calculates the file hash and checks supported indicators.
6. The risk engine evaluates the findings.
7. The application shows the risk level and the reasons.
8. The Attachment Threat Story shows the available events and their sources.
9. The user reviews the findings and the quarantine simulation, where available.
10. The user views or exports a security report, where supported.

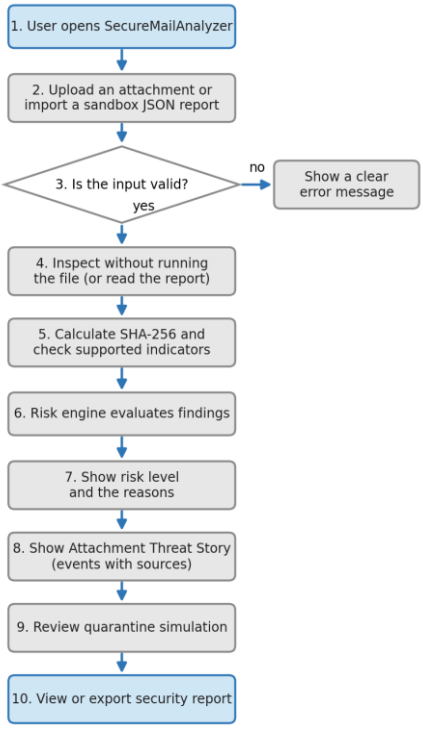


Figure 1: Main application workflow

16. USE CASES

Figure 2 shows the main use cases. The table that follows describes each one. Every use case is marked to verify until it is confirmed in the application.

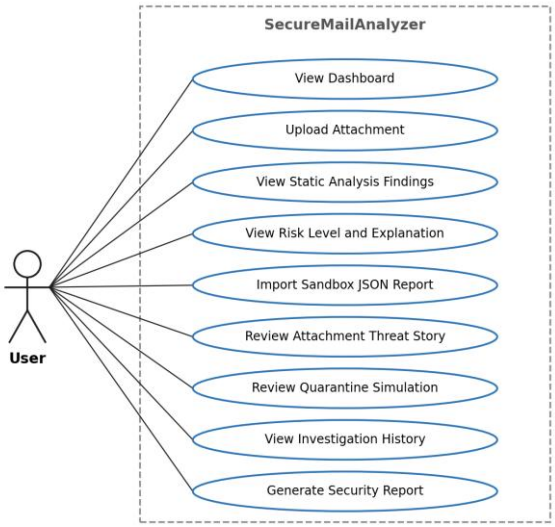


Figure 2: Main use cases

ID	Use case	User	Preconditions	Main steps	Expected result
UC-01	Upload attachment	User	Application is running; a supported file is ready	1. Open the upload screen. 2. Choose a file. 3. Submit.	File is accepted, or a clear error is shown.
UC-02	Run static analysis	User	A file has been accepted	1. Start analysis. 2. Wait for the result. 3. Read the findings.	SHA-256 and findings are shown.
UC-03	Review risk assessment	User	Analysis is complete	1. Open the result. 2. Read the risk level. 3. Read the reasons.	Risk level, reasons and recommendations are shown.
UC-04	Import sandbox JSON	User	A sandbox JSON report is available	1. Open the import screen. 2. Choose the JSON file. 3. Submit.	Indicators are extracted, or an error is shown.
UC-05	Review threat story	User	An investigation exists	1. Open the investigation. 2. Open the timeline. 3. Check each event source.	Events appear in time order with source labels.
UC-06	Simulate quarantine	User	A risk result exists	1. Open the quarantine view. 2. Review the decision and reason.	A simulated status is shown with a notice that no real isolation occurs.
UC-07	Generate report	User	An investigation exists	1. Choose the report option. 2. Select a supported format. 3. Download.	A report file is created, or an error is shown.
UC-08	View history	User	Earlier investigations were saved	1. Open the history screen. 2. Select a record.	Past investigation details are shown.

17. INPUT AND OUTPUT REQUIREMENTS

17.1 Inputs

Input	Description	Validation rules
Email attachment	A file uploaded by the user.	Supported types and the size limit must be confirmed from the code. Empty files, unsupported types and unsafe filenames are rejected.
Sandbox JSON report	An existing report in JSON format.	Must be valid JSON with the structure the application supports. The exact structure must be confirmed from the code.

17.2 Outputs

- File details, including the SHA-256 hash and the result of the signature check

- Findings and suspicious indicators in plain language
- Risk score and risk level, with the indicators that caused them
- Attachment Threat Story events, each with a source label
- Quarantine simulation status and reason
- Investigation history entries
- Security report in the formats that are confirmed to work
- Clear error messages for invalid input

No supported file types, size limits or report formats are stated here because they were not available. They must be added after checking the application.

18. RISK SCORING AND CLASSIFICATION

The risk engine evaluates the indicators found during analysis and calculates a risk score. The score is then linked to a risk level. Figure 3 shows the process.

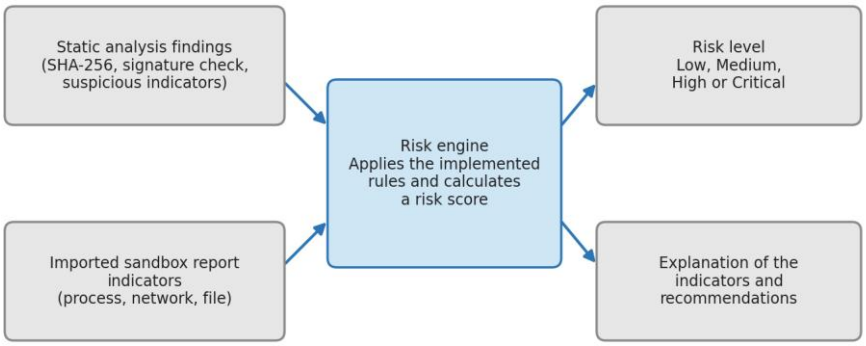


Figure 3: Risk assessment process

The scoring formula, the weight of each rule and the score ranges for each level were not available when this document was prepared. They must be copied from the source code. They are not stated here so that no wrong values are published.

Risk level	Meaning	Score range
Low	Few or no concerning indicators were found. This does not mean the file is safe.	Requires verification
Medium	Some suspicious indicators were found. A manual review is advised.	Requires verification
High	Strong indicators of risky content or behaviour were found.	Requires verification
Critical	Several strong indicators were found.	Requires verification

Risk scoring is based only on the indicators available. It can give false positives (a safe file marked risky) and false negatives (a harmful file missed).

19. ATTACHMENT THREAT STORY AND EVIDENCE SOURCES

The Attachment Threat Story is a timeline of investigation events. It helps the user see what was found and when it was recorded. It is useful because it turns separate findings into one clear story.

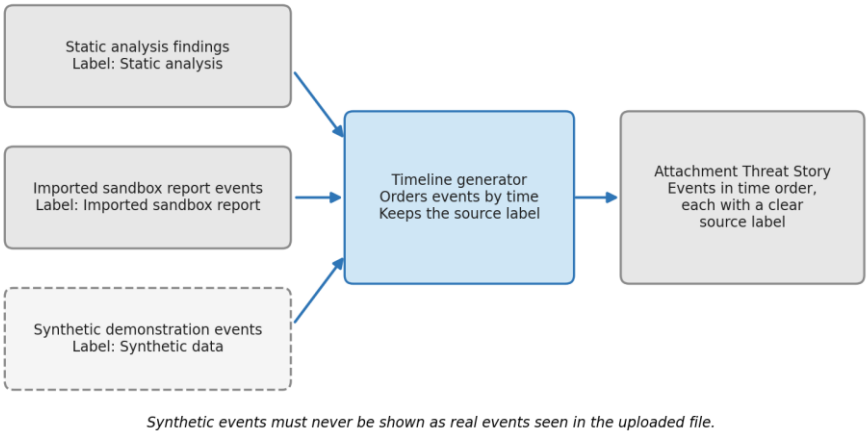


Figure 4: Investigation timeline flow

Every event must show where it came from. There are three sources:

Source	Meaning	How it must be shown
Static analysis	Found directly by checking the uploaded file without running it.	Labelled as a static analysis finding.
Imported sandbox report	Read from a sandbox report that was imported. SecureMailAnalyzer did not run the file.	Labelled as an imported sandbox report event.
Synthetic data	Created only for demonstration or testing.	Clearly labelled as synthetic. It must never look like a real observation.

If the source of an event is not clear to the user, the requirement is not met.

20. SECURITY AND PRIVACY

The following rules apply to the application. They are requirements. Whether each one is already met must be checked in the code and in testing.

ID	Requirement	Verification
SR-01	Uploaded attachments shall never be executed by the application.	Required rule

ID	Requirement	Verification
SR-02	Suspicious scripts and macros shall not be run.	Required rule
SR-03	Uploaded files shall not be opened automatically in external applications.	Required rule
SR-04	Archive contents shall not be executed or extracted unsafely.	Required rule
SR-05	File size, file type, filenames and imported JSON shall be validated.	Required rule
SR-06	Unsafe file paths and path traversal shall be prevented.	Required rule
SR-07	Untrusted text (such as filenames and report content) shall be handled safely in the interface and in generated reports.	Required rule
SR-08	Uploaded files and investigation data shall be handled according to the actual storage design. Where files and records are kept must be confirmed.	To verify
SR-09	The local server should stay limited to the intended local environment where configured.	To verify
SR-10	Safe sample files and synthetic reports shall be used for demonstrations.	Required rule

Notice: this prototype does not replace professional security products. Its findings are indicators, not proof.

21. DATABASE AND DATA MANAGEMENT

The project uses SQLite, a small database stored in a file. The table names, column names and relationships were not available for this document, so none are listed. They must be added after checking the database design.

Information stored	Purpose	Verification
Investigation records	Keep a record of each analysis.	To verify
Analysis details	Keep findings, hash and risk result.	To verify
Timeline events	Keep the Attachment Threat Story and event sources.	To verify
Quarantine status	Keep the simulated decision and reason.	To verify

A database diagram should be added only after the real design is confirmed. The database must also be handled safely, which means using safe queries and handling database errors without crashing.

22. USER INTERFACE REQUIREMENTS

The project description mentions these screens: dashboard, file analysis, investigation timeline, quarantine view and reports (To verify). The interface shall follow these rules:

- Navigation shall be clear and consistent on every screen.
- Risk levels shall be easy to read and shall not depend on colour alone.
- Error messages shall be helpful and written in simple language.

- The layout shall be consistent across screens.
- Simulated or imported data shall be clearly labelled.

23. ERROR HANDLING AND VALIDATION

The table shows the expected handling for each situation. It is recommended behaviour. Whether it is implemented must be checked.

Situation	Expected handling	Status
Empty upload	Reject and ask the user to choose a file.	To verify
Unsupported file	Reject and say which types are supported.	To verify
Invalid filename	Reject or replace with a safe name.	To verify
Oversized file	Reject and state the size limit.	To verify
Malformed sandbox JSON	Reject and explain that the report is not valid.	To verify
Failed analysis	Show an error and keep the application running.	To verify
Database error	Show a clear message without exposing internal details.	To verify
Report generation failure	Show a message and allow the user to try again.	To verify
Unexpected server error	Show a general message and record the error safely.	To verify

24. TESTING AND TEST CASES

No test results were available. Every actual result and status is therefore marked Not yet verified. They must be filled in after running the tests with safe sample files. Expected results do not include exact scores because the scoring rules were not available.

ID	Feature	Test input or steps	Expected result	Actual result	Status
TC-01	File upload	Upload a supported safe sample file.	File is accepted.	Not yet verified	Not yet verified
TC-02	Hash calculation	Analyse a file, then calculate its SHA-256 with a separate tool.	Both hashes are the same.	Not yet verified	Not yet verified
TC-03	Signature mismatch	Upload a safe file whose extension was changed.	A mismatch finding is shown.	Not yet verified	Not yet verified
TC-04	Suspicious indicators	Upload a safe sample that contains a supported indicator.	The indicator is listed with an explanation.	Not yet verified	Not yet verified
TC-05	Archive handling	Upload a safe archive sample.	Contents are not run or extracted unsafely.	Not yet verified	Not yet verified
TC-06	JSON parsing	Import a valid synthetic sandbox report, then a broken JSON file.	Indicators are extracted from the valid file; the broken file gives a clear error.	Not yet verified	Not yet verified

ID	Feature	Test input or steps	Expected result	Actual result	Status
TC-07	Risk scoring	Compare a clean sample with a sample having several indicators.	The second result has an equal or higher risk level, with reasons.	Not yet verified	Not yet verified
TC-08	Event sources	Open the timeline for imported and demonstration data.	Each event shows its source; synthetic events are clearly labelled.	Not yet verified	Not yet verified
TC-09	Quarantine simulation	Open the quarantine view for a result.	Status and reason are shown with a simulation notice.	Not yet verified	Not yet verified
TC-10	History	Complete two analyses, then open history.	Both investigations are listed.	Not yet verified	Not yet verified
TC-11	Report download	Download a report in a supported format.	The file opens and shows the expected content.	Not yet verified	Not yet verified
TC-12	Invalid uploads	Submit an empty file, an unsupported type and an oversized file.	Each is rejected with a clear message.	Not yet verified	Not yet verified
TC-13	Unsafe filename	Upload files named with path characters and with HTML characters.	Names are made safe and shown as plain text; no script runs.	Not yet verified	Not yet verified
TC-14	Failed analysis	Cause a failure with a safe corrupt file.	An error is shown; the application keeps working.	Not yet verified	Not yet verified

25. SOFTWARE AND HARDWARE REQUIREMENTS

Minimum system specifications were not measured, so none are stated. All items are To verify.

- Windows (development environment), Visual Studio Code and Windows PowerShell
- Python for the backend and Node.js for the frontend (versions to be confirmed)
- The packages listed in the project files, and a current web browser
- A computer able to run Python, Node.js and a browser

26. LIMITATIONS

- Static analysis cannot find every possible threat.
- A risk score is not proof that a file is safe or harmful.
- Detection depends on the supported file formats and the implemented rules.
- Imported sandbox reports are only as useful as the information they contain.
- Quarantine is simulated. No real isolation takes place.
- The application is a prototype and does not replace a professional malware analysis system.

27. FUTURE ENHANCEMENTS

These are ideas for future work. They are not part of the current application.

- More detection rules and supported file types
- YARA rule integration
- Real sandbox integration in a properly isolated environment
- Stronger access control, such as user accounts and roles
- Improved and more detailed reports
- Hash checks against threat intelligence services

28. EXPECTED OUTCOMES

After using SecureMailAnalyzer, a user should be able to:

- Inspect a supported attachment without running it
- See its SHA-256 hash and the suspicious signs found
- Understand the risk level and the reasons behind it
- Import a sandbox report and understand what it shows
- Read the Attachment Threat Story and know the source of each event
- Review a simulated quarantine decision
- Look back at earlier investigations and create a security report

These outcomes are expected once all modules are confirmed to work.

29. CONCLUSION

SecureMailAnalyzer helps users inspect email attachments safely and understand the results. It combines file analysis, risk assessment, imported sandbox information, an evidence-based timeline, quarantine simulation and reports in one web application.

Its main value is clear explanation. It shows why a file looks suspicious and where each piece of evidence comes from. It is a prototype and must be used with care, because its results are indicators and not proof.

30. GLOSSARY

Term	Meaning
Attachment	A file sent with an email.
Malware	Software made to cause harm.
Static analysis	Checking a file without running it.
SHA-256	A method that creates a unique fingerprint (hash) of a file. If the file changes, the hash changes.

Term	Meaning
Risk score	A number that shows how risky a file looks, based on the indicators found.
Sandbox	A safe, separate environment where a file can be run and watched.
JSON	A simple text format used to store and share data.
Quarantine	Keeping a suspicious file apart so it cannot cause harm. In this project it is only simulated.
False positive	A safe file that is wrongly marked as risky.
False negative	A harmful file that is wrongly marked as safe or low risk.
API	A way for two programs to talk to each other. Here, the frontend talks to the backend.

APPENDIX A. ITEMS TO VERIFY BEFORE SUBMISSION

Check these items against the code, database and running application, then update the labels.

- Which modules and screens are fully implemented
- Supported file types, the upload size limit and the exact static indicators
- The risk scoring formula, rule weights and score ranges
- The sandbox JSON structure, synthetic event labelling and report formats
- Database tables and columns, and whether any login or role system exists
- Python and Node.js versions, the package list and real test results (Section 24)